# Matter.js: Implementing Hot Restart

Adan Rodriguez

September 5, 2026

## Contents



# 1 Background: where this left off

The earlier note in this document (see the previous draft) observed the following behavior when saving a file during development:

> *On save, the compiler rebuilds and* `webpack-hot-middleware` *pushes the update over the webpack HMR SSE stream. The client checks whether any module in the update chain called* `.accept()` *— none did — so it logs* `"[HMR] ... do not know how to hot reload themselves ... (Full reload needed)"` *and stops. Because* `reload` *is* `false`*, it does not auto-refresh either. The page sits there unchanged until a human manually hits refresh.*

Two remediation paths were identified: (1) wire up real `module.hot.accept()` / `module.hot.dispose()` handling around the render loop, being careful not to stack duplicate loops on repeated updates, or (2) a simpler `reload=true` query-string override that forces a full page reload on every save. This document implements path (1), specifically the *hot restart* variant of it: on every accepted update, the previous simulation is fully torn down and a fresh one is built, rather than trying to preserve in-flight physics state across edits.

# 2 The change

## 2.1 Before

```
const render = (context, engine) => {
  clearCanvas(context, CANVAS_WIDTH, CANVAS_HEIGHT);
  drawBodies(context, engine.world.bodies);
  requestAnimationFrame(() => render(context, engine));
};

const main = (() => {
  const { Engine, Runner, Bodies, Composite } = Matter;

  const engine = Engine.create();
  Composite.add(engine.world, createWorldBodies(Bodies));

  const canvas = createCanvas(CANVAS_WIDTH, CANVAS_HEIGHT);
  const context = getContext2d(canvas);
  mountCanvas(canvas);

  const runner = Runner.create();
  Runner.run(runner, engine);

  render(context, engine);
})();
```

`main` is an anonymous, self-invoking function expression (IIFE). It runs exactly once, at module-evaluation time, and everything it creates (`engine`, `canvas`, `runner`) is scoped inside its closure — nothing outside can reach them. `render` recurses through `requestAnimationFrame` without ever storing the frame id it receives, so the loop cannot be cancelled from outside itself.

## 2.2 After

```
let engine, runner, canvas, rafId;

const render = (context, engine) => {
  clearCanvas(context, CANVAS_WIDTH, CANVAS_HEIGHT);
  drawBodies(context, engine.world.bodies);
  rafId = requestAnimationFrame(() => render(context, engine));
};

const start = () => {
  const { Engine, Runner, Bodies, Composite } = Matter;

  engine = Engine.create();
  Composite.add(engine.world, createWorldBodies(Bodies));

  canvas = createCanvas(CANVAS_WIDTH, CANVAS_HEIGHT);
  const context = getContext2d(canvas);
  mountCanvas(canvas);

  runner = Runner.create();
  Runner.run(runner, engine);

  render(context, engine);
};

const stop = () => {
  Matter.Runner.stop(runner);
  cancelAnimationFrame(rafId);
  canvas.remove();
};

start();

if (module.hot) {
  module.hot.dispose(stop);
  module.hot.accept();
}
```

### 2.3 Logical explanation of each change

**Module-level state**
: `engine`, `runner`, `canvas` and `rafId` move from `const`s trapped inside the IIFE's closure to `let` bindings at module scope. This is the prerequisite for everything else: a `dispose` callback that runs later, outside `start`, must be able to see and act on the exact instances `start` created.

`main` → `start`
: The IIFE becomes a named, callable function. An IIFE runs once by construction; a restart strategy needs to invoke setup again, so setup has to be a function with a name, not a value produced by immediately invoking one.

`rafId` **capture**
: `render` now assigns its own `requestAnimationFrame` return value to the module-level `rafId` on every frame. Previously this id was discarded, so there was no way to stop the animation loop from outside; now `stop` can call `cancelAnimationFrame(rafId)` against the most recent pending frame.

`stop`
: A new function performing full, unconditional teardown: stop the physics `Runner` (halts the fixed-timestep physics ticks), cancel the pending animation frame (halts the draw loop), and remove the mounted `<canvas>` from the DOM. Note that `Runner` is referenced as `Matter.Runner` here rather than the destructured local from `start` — the destructured binding only exists inside `start`'s scope, so `stop` must go through the `Matter` namespace import directly.

`module.hot.dispose(stop)`
: Registers `stop` as the cleanup callback for *this instance* of the module. Webpack's HMR runtime guarantees this callback runs immediately before the replacement module executes, and never any earlier — so the old simulation is torn down exactly once, right before the new one starts.

`module.hot.accept()`
: Called with no arguments, this self-accepts updates to *this* module. It tells the HMR runtime "when this module (or anything importing it up to here) changes, don't bubble the update further up the module graph or fall back to a full page reload — just re-execute this module in place." Because `start()` is called unconditionally at the bottom of the file, re-executing the module is what performs the actual restart: a brand new `Engine`, brand new bodies at their original coordinates from `createWorldBodies`, a brand new canvas.

This is a *restart*, not a state-preserving *reload*: no attempt is made to save body positions, velocities, or any other simulation state across an edit. Every accepted save produces a scene identical to a hard refresh, but without the browser navigating away, reconnecting the dev-server socket, or re-fetching the full HTML document.

## 3 Execution model

### 3.1 Cold start (first page load)

1. The browser requests `/`; Express responds with `dist/index.html` (generated by `HtmlWebpackPlugin`, which already references `main.bundle.js`).
2. The browser requests `main.bundle.js`; the webpack runtime boots, evaluates `src/index.js` for the first time.
3. At the bottom of the module, `module.hot` exists (because `HotModuleReplacementPlugin` is active and the entry includes `webpack-hot-middleware/client`), so `module.hot.dispose` and `module.hot.accept` are called — but `stop` has not run yet, since nothing has been replaced.
4. `start()` has already run synchronously just above that block, so the canvas is mounted and the render/physics loops are live.

### 3.2 Hot restart (save while the dev server is running)

1. A source file changes on disk.
2. Webpack's compiler (running in watch mode inside `webpack-dev-middleware`) recompiles only the affected modules and computes an update chain.
3. `webpack-hot-middleware` pushes a `hash` event, then the browser client requests the small `*.hot-update.json` manifest and the `*.hot-update.js` chunk containing the new module code.
4. The webpack runtime in the browser walks the module graph looking for an accept handler. It finds `module.hot.accept()` was called by `src/index.js` itself, so the update chain terminates there — no reload is needed, and the update does not have to bubble to `index.js`'s importers (there are none in this app).
5. Before swapping in the new module factory, the runtime invokes every registered `dispose` callback for modules being replaced — here, `stop()`: the `Runner` halts, the pending animation frame is cancelled, the old `<canvas>` is removed from the DOM.
6. The new module code executes in place of the old one, which means `start()` runs again: fresh `Engine`, fresh bodies, fresh canvas mounted, fresh runner and render loop — and the new `module.hot.dispose`/`accept` pair is registered for the *next* save.

## 4 How webpack talks to the browser

Two independent express middlewares are layered in `app.js` to make this possible: `webpack-dev-middleware` handles serving compiled assets from memory, and `webpack-hot-middleware` handles notifying the browser when new assets are ready.

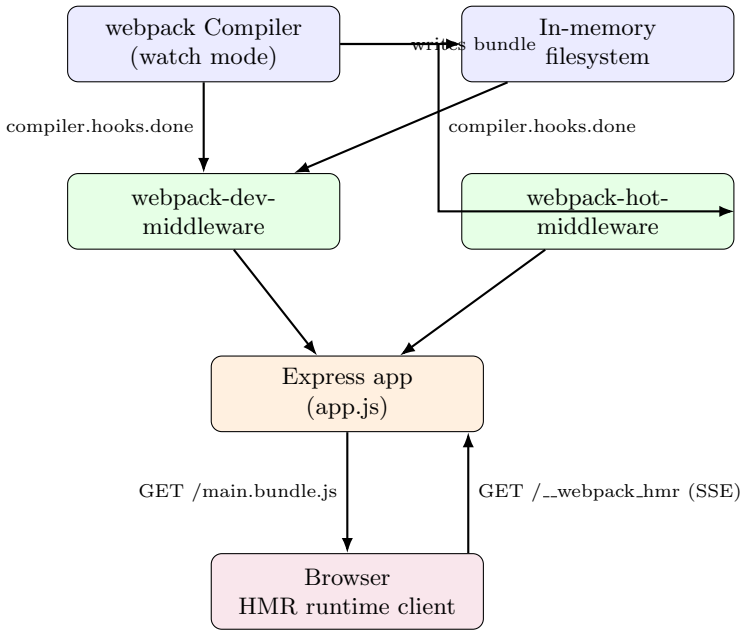


Figure 1: Two middlewares wrap the same compiler instance: one serves files, the other streams update notifications.

- **webpack-dev-middleware** owns the compiler's watch cycle. It keeps the latest build in an in-memory filesystem (no assets are written to `dist/` on disk in the request path) and answers ordinary asset requests (`main.bundle.js`, `index.html`) straight out of that memory, always the freshest build — this is why the server does not need restarting when source changes.

- **webpack-hot-middleware** listens to the same compiler's `done` hook. On every successful recompilation it opens/keeps a Server-Sent-Events stream at `/__webpack_hmr` and pushes a small JSON event (essentially: *"new build hash X is ready"*).
- The browser-side half of `webpack-hot-middleware` (`webpack-hot-middleware/client`, injected because it is prepended to the `main` entry in `webpack.config.js`) is what actually holds that SSE connection open, requests the `hot-update.json`/`hot-update.js` pair when notified, and hands them to webpack's HMR runtime (`module.hot.{accept, dispose}`, `__webpack_require__.hmrM`, etc.) to apply.

# 5 How Express talks to webpack

`app.js` does not run webpack as a separate CLI process; it imports the webpack config directly and drives the compiler in-process:

```
const compiler = webpack(webPackConfig);
const app = express();

app.use(webpackDevMiddleware(compiler));
app.use(webpackHotMiddleware(compiler));
app.use(express.static(path.join(__dirname, 'dist'), { index: false }));

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'dist', 'index.html'));
});
```

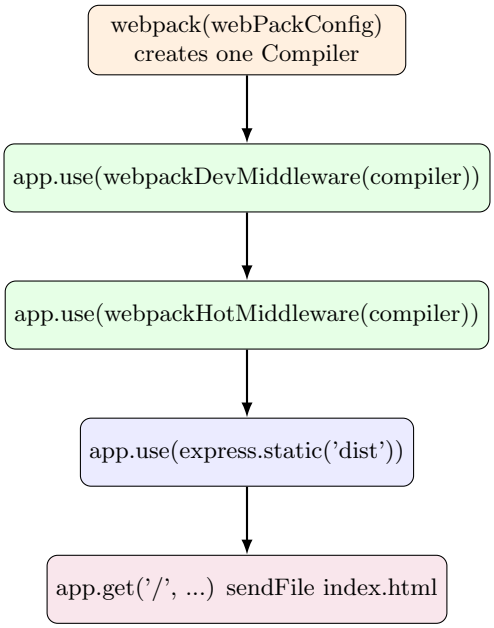


Figure 2: Express middleware order in `app.js`. Both HMR middlewares share the *same* compiler instance, which is what lets them coordinate on one build.

`webpack(webPackConfig)` only builds a `Compiler` object; it does not compile anything by itself. Handing that one compiler to both `webpack-dev-middleware` and `webpack-hot-middleware` is what lets them stay in sync: the dev-middleware starts the compiler's watch loop and exposes its output, and the hot-middleware taps into the lifecycle hooks of that exact same compiler to know when a new build is ready to announce. If two separate `webpack(...)` calls were used instead, the two middlewares would be watching unrelated builds and would drift out of sync. `express.static` and the explicit `/` route are the fallback path for

anything the dev-middleware does not intercept, and only matter for the very first request (`index.html`) since dev-middleware already answers for `main.bundle.js` and the hot-update files.

# 6 Sequence diagram: a hot-restart round trip

# 7 Why this counts as a *restart* and not a *reload*

Two different things are commonly called "hot reload," and this codebase now implements the simpler of the two:

| | **State-preserving reload** | **Hot restart (this change)** |
|---|---|---|
| Simulation state across an edit | Kept (e.g. bodies keep their current position/velocity) | Discarded; scene resets to `createWorldBodies`' initial layout |
| `dispose` responsibility | Selectively serialize state into `module.hot.data` | Unconditionally stop everything: runner, rAF, DOM node |
| Complexity | Higher — must reconcile old/new state shape when code changes | Lower — next `start()` call is identical to a cold boot |
| Full page reload / socket reconnect | No | No |

Extending this into a true state-preserving reload later would only require `stop` to stash whatever should survive (e.g. body positions) onto `module.hot.data`, and `start` to read it back out if present — the `accept`/`dispose` wiring itself would not change.

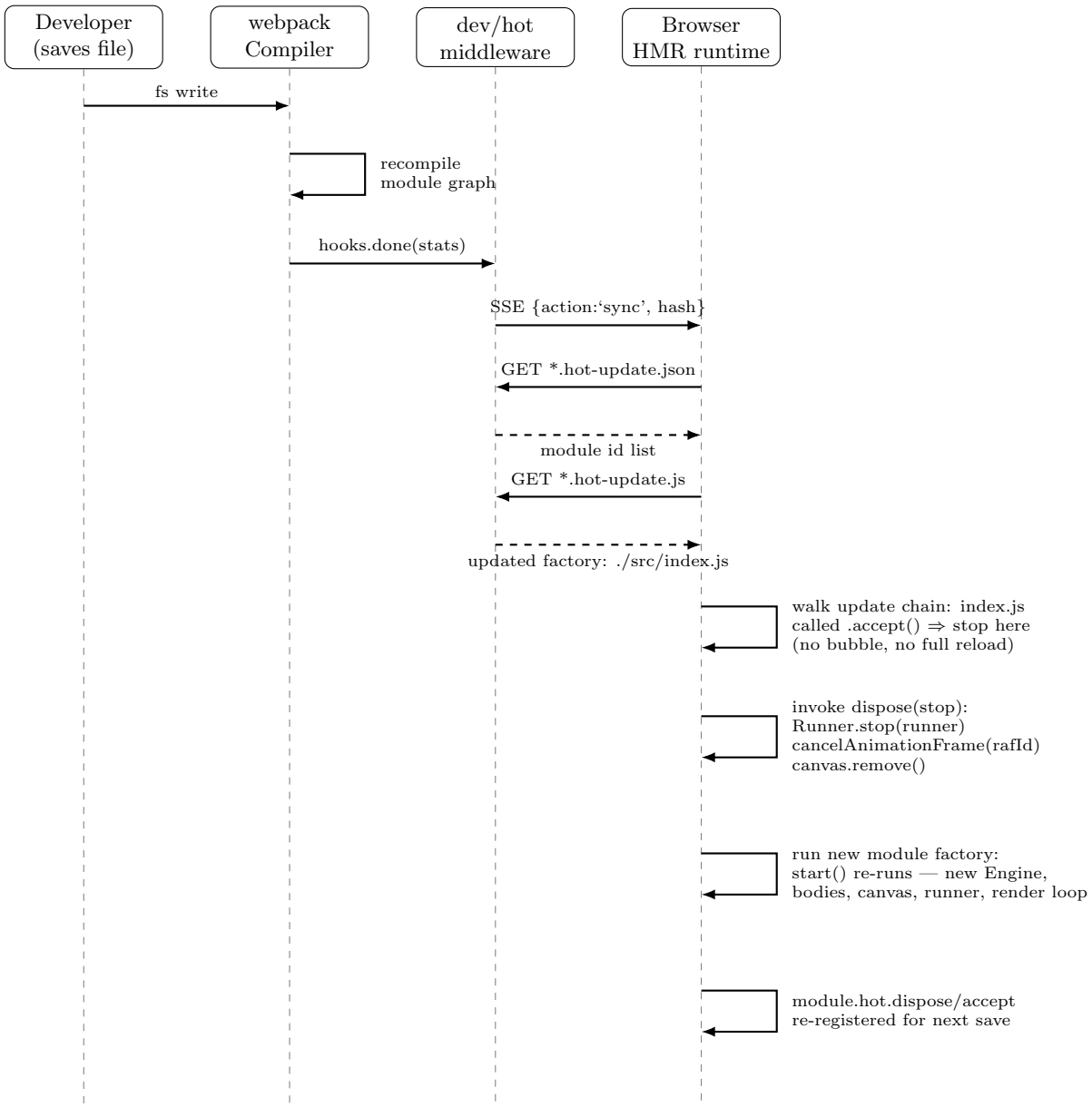


Figure 3: End-to-end timeline from file save to the DOM showing a freshly restarted simulation.